

=== PROJETO RADAR COORDINATOR - DETALHAMENTO DE ENGENHARIA ===
DOCUMENTO DE ESPECIFICACAO TECNICA COMPLETA

SECTION: 1. VISAO GERAL E ARQUITETURA REATIVA

=====

O Projeto Radar Coordinator é uma plataforma integrada de alta fidelidade projetada especificamente para otimizar, gerenciar e automatizar fluxos de trabalho de entregadores que operam em múltiplos aplicativos simultaneamente.

O ecossistema é baseado em uma arquitetura de triade reativa que conecta o painel do motorista (disponibilizado em index.html via React Standalone) a um banco de dados em tempo real no Google Firebase Firestore, que por sua vez sincroniza dados instantaneamente com serviços nativos em segundo plano rodando em Kotlin no celular Android do usuário.

Através desta sincronização em tempo real, quaisquer alterações de regras ou parâmetros efetuadas no painel Web são traduzidas e aplicadas imediatamente aos algoritmos de execução nativa do Android.

SECTION: 2. SCHEMA DETALHADO DO FIREBASE FIRESTORE

=====

Para garantir a perfeita integridade operacional e sincronização entre o aplicativo e a Web, o documento 'RadarSettings' no Firestore contém campos fundamentais mapeados rigorosamente.

Os campos principais são:

- chainDeliveriesMode (Boolean): Ativa o agrupamento sequenciado de entregas pendentes.
- isGhostSequenceEnabled (Boolean): Ativa a IA de Roteamento Preditivo Sequência Fantasma.
- ghostSequenceAggressiveness (String): Níveis 'CONSERVADOR', 'EQUILIBRADO' ou 'AGRESSIVO'.
- ghostSequenceTrafficWeight (Double): Peso do trânsito na ponderação de custos da rota.
- ghostSequenceLatencyWeight (Double): Peso da latência local no recálculo.
- systemHealthScore (Int): Pontuação de saúde de 0 a 100 gerada ativamente pelo APK.

- activeAnomalies (List): Lista de inconsistências ativas detectadas (ex: GPS_OFFLINE, GHOST_IDLE).
- jarvisVoiceState (String): Estado atual da voz (IDLE, LISTENING_WAKEWORD, LISTENING_COMMAND, PROCESSING, SPEAKING).

SECTION: 3. SERVICOS NATIVOS ANDROID

=====

O aplicativo nativo executa três serviços centrais de engenharia:

1. RadarCoordinatorService: Concentra os estados de decisão, telemetria e o diagnóstico de integridade do sistema que é executado de forma recorrente a cada 30 segundos em segundo plano.
2. RadarAccessibilityService: Monitora ativamente os textos e botões das interfaces dos apps de entrega suportados (Uber, iFood, Rappi), extraíndo valores monetários, quilometragens e endereços com OCR espacial e realizando cliques semânticos automáticos.
3. VoiceInputManager e VoiceManager: Fornecem controle completo por voz. Com o modo Jarvis, o sistema fica em escuta passiva aguardando palavras de ativação ('Jarvis' ou 'Ok Google'). Ao anunciar ofertas de entrega por voz, o sistema habilita automaticamente o reconhecimento de comandos imediatamente após a fala, permitindo respostas rápidas como 'aceitar' ou 'recusar'.

SECTION: 4. ALGORITMOS DE ROTEAMENTO E INTEGRIDADE

=====

A IA Sequência Fantasma (Ghost Sequence) calcula em tempo real o custo de desvios e rotas inteligentes pela fórmula:

$$\text{Custo} = \text{Distancia} * (1.0 + (\text{Fator_Trafego} * \text{Fator_Agressividade} * \text{trafficWeight}) + (\text{latencyWeight} * 0.2))$$

O Loop de Saúde do Sistema (System Health Pulse) roda a cada 30 segundos no Android. Se inconsistências forem encontradas, pontos são deduzidos da pontuação máxima de 100 e gravados no Firestore:

- Ausência de sinal de GPS deduz 30 pontos (anomalia: GPS_OFFLINE).
- Sinal de localização estagnado por mais de 1 minuto deduz 15 pontos (GPS_STALE).

- Congestionamento severo de transito deduz 10 pontos (TRAFFIC_CONGESTION).
- IA ativada porem sem rotas pendentes deduz 5 pontos (GHOST_IDLE).

SECTION: 5. GRADE NEURAL DE MULTIPLEXACAO (4 TELAS)

=====

A Multiplexadora Neural e um recurso avançado do painel que divide a tela em uma matriz 2x2 simulando 4 ambientes paralelos ativos:

- VM_01: UBER_DRV - Simulador e hook do aplicativo Uber Driver.
- VM_02: IFOOD_LOG - Interceptador e parser do iFood Entregador.
- VM_03: G_MAPS_SYS - Telemetria e analise de transito do Google Maps.
- VM_04: RAPPI_LOG - Simulador de logs e pacotes stealth do Rappi.

O painel visual no index.html reflete e sincroniza os estados de voz do Jarvis vindo do Android nativo com mudancas de cores e animacoes correspondentes aos estados de IDLE, LISTENING, PROCESSING, e SPEAKING.

SECTION: 6. PROMPT DE INGESTAO PARA IA EXTERNA

=====

Cole este prompt na outra IA de analise:

'Voce esta recebendo o arquivo de especificacoes completas do Projeto Radar Coordinator. Analise as relacoes entre a interface de visualizacao paralela (4 telas), o banco de dados Firebase Firestore e os servicos em Kotlin (RadarCoordinatorService e RadarAccessibilityService). Foque em otimizar as regras de negocio do Roteamento Fantasma, tratamento defensivo de tipos numericos no Kotlin (evitando ClassCastException) e no fluxo de voz com auto-wake.'